

GIT

What Is Git?

- Git is a free open-source distributed version control system designed to handle everything from small to extensive projects quickly and efficiently.

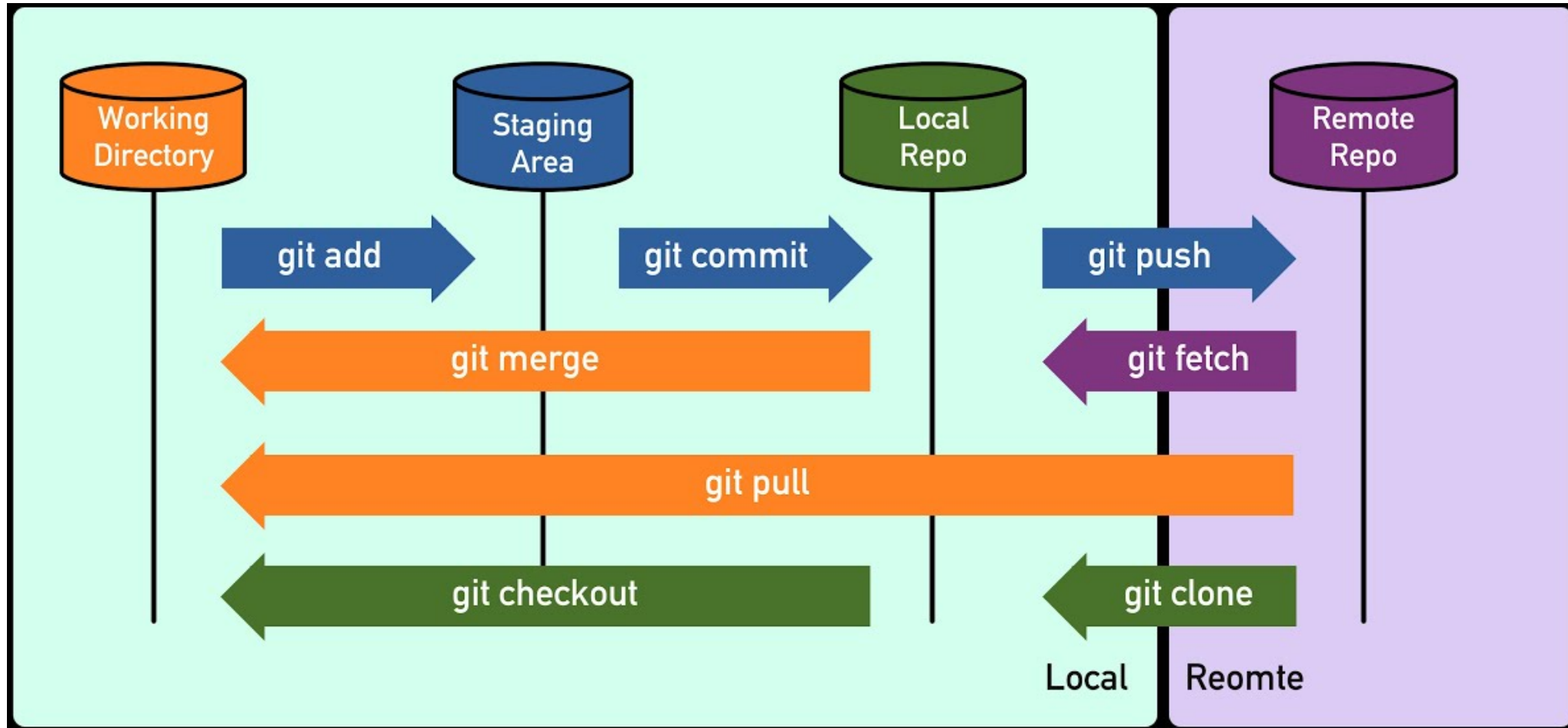
What Is VCS?

- Version control system generally allows users to revisit earlier versions of the files, compare changes between versions, undo changes & a whole lot more. There is lot's of Version control systems(Subversion, CVS), etc. Git is just like one of them.

Why Use Git?

1. Back To Old Versions.
2. Compare Versions Of a Project.
3. Collaborate & share changes.
4. Combine Changes.
5. Tracking Changes Across Multiple Files.
6. Revert to previous versions.

How Git Actually Works



Installing Git

Installing Git

- Download the latest Git from this url <https://git-scm.com/downloads>
- Once the download is complete, run the installer. You can use the default settings for most options.
- Finish the installation process. Once completed, you can launch Git Bash, a command-line terminal, from the Start menu.
- **Verify Installation:**
 - Open Git Bash and type the following command to verify the installation ***git --version***

Git Config

Accessing User Name

git config user.name

Setting User Name

git config --global user.name "HuXn WebDev"

Accessing User Email

git config user.email

Setting User Email

git config --global user.email test@test.com

Git Repo

What Is A Git Repo?

- Anytime we want to use git with a project, app, etc. we need to create new git repository, we can have as many repos on our machine as needed, all with separate histories and content.

Git Status

Git Status

- Git status is a command used in Git to display the current state of the working directory and the staging area (index). It shows which files are modified, staged, or untracked since the last commit. This command is essential for understanding the changes made to the repository and determining what needs to be committed or further modified.

Git Status

Show Current Status of Working Directory and Staging Area:

`git status`

Display Changes Not Yet Staged for Commit:

`git status --short`

Show Detailed Status with Color Coding:

`git status --verbose`

Show Untracked Files:

`git status --untracked-files`

Show Status of Specific File:

`git status filename.txt`

Git Init

Git init

- The `git init` command is used to create a new Git repository. It can either convert an existing, unversioned project into a Git repository or initialize a new, empty repository. When executed, `git init` creates a `.git` subdirectory in the current working directory, containing all the necessary Git metadata for the new repository, including objects, refs, and template files. Additionally, a `HEAD` file is created, pointing to the currently checked-out commit.

Git Init

Initialize a New Git Repository:

- Creates a new .git directory to track version control.

```
git init
```

Initialize a Git Repository in a Specific Directory:

- Creates a new repository in the specified directory.

```
git init <directory>
```

Initialize Repository with Default Branch Name:

- Sets the initial branch name (e.g., main or master).

```
git init --initial-branch=<branch-name>
```

Git Add

Git Add

- The git add command adds a change in the working directory to the staging area.
- It's a way of telling Git (Please include this changes in our next commit)

Git Add

Add Specific Files

```
git add filename.txt  
git add directory/
```

Add All Changes (excluding deletions)

```
git add .
```

Add All Changes (including deletions):

```
git add -A
```

Add Files Matching a Pattern:

```
git add '*.txt'
```

Add Part of a File (interactive):

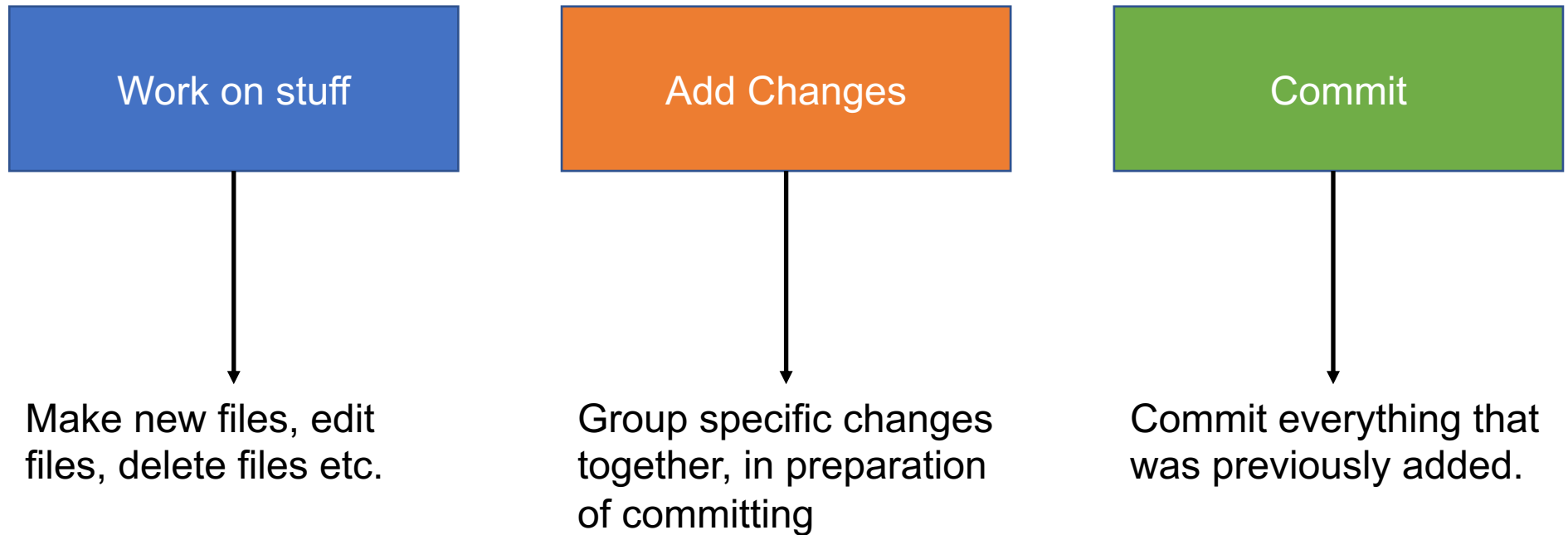
```
git add -p filename.txt
```

Git Commit

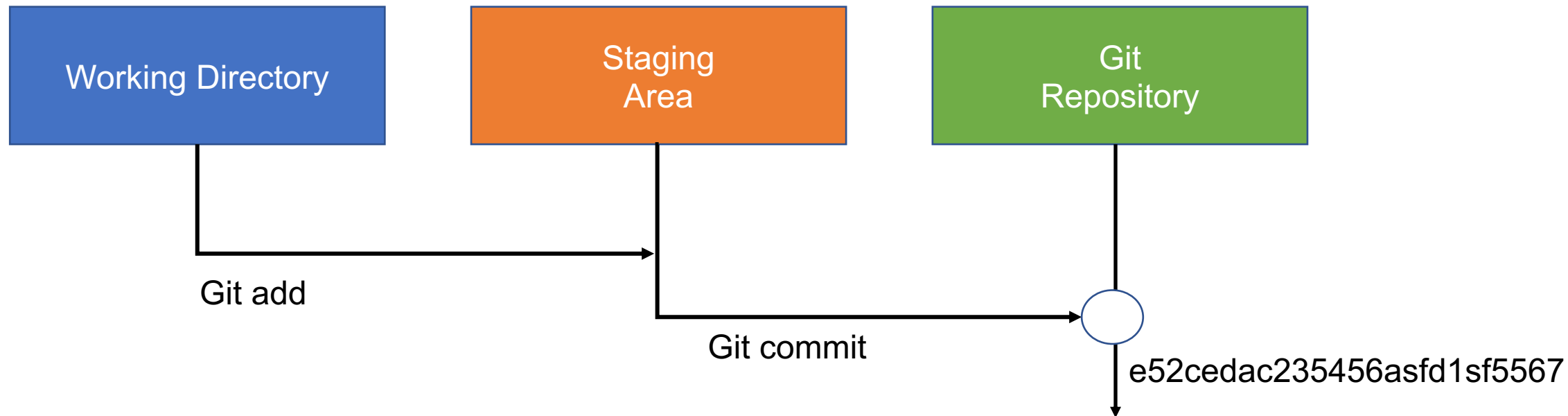
What is a Git Commit

- A commit, or "revision", is an individual change to a file(s). It's like when you save a file, except with Git, every time you save it creates a unique ID (a.k.a. the "SHA" or "hash") that allows you to keep record of what changes were made when and by who.

Git Commit



Git Commit



Git Commit

- We use the `git commit` command to commit changes from the staging area.
- when making a commit, we need to provide a commit message that summarizes the changes and work snapshotted in the commit.

-m flag

- The -m flag, which stands for message, is used to add a commit message to a commit.

Git Commit

```
git commit -m "Your commit message"
```

Git Fetch

Git Fetch

- git fetch is used to download updates from a remote repository to your local repository without merging those changes into your current branch.
- It updates your remote-tracking branches, allowing you to see what others have done before you integrate their changes.

Fetch Updates from All Remotes

```
git fetch
```

Download updates from a specific remote repository and Specific Branch

```
git fetch <remote>  
git fetch origin  
git fetch <remote> <branch>  
git fetch origin main
```

Git Pull

Git Pull

- git pull is used to update your local branch with changes from a remote repository.
- It is essentially a combination of **git fetch** (which updates your local copy of the remote branch) and **git merge** (which merges the fetched changes into your current branch).

fetch and merge changes from the remote branch into your current branch

```
git pull
```

Pull from a Specific Remote and Branch:

```
git pull <remote> <branch>  
git pull origin main
```

Git Push

Git Push

- git push is used to upload your local branch commits to a remote repository. It updates the remote branch with the changes from your local branch, making your updates available to others.

Push to Default Remote:

```
git fetch
```

Push to a Specific Remote and Branch

```
git push <remote> <branch>  
git push origin main
```

Force Push (Caution): If you need to overwrite the remote branch (e.g., after rewriting history), use

```
git push --force
```

Git Clone

Git Clone

- git clone is used to create a local copy of a remote repository. This command copies the entire repository, including its history, branches, and commits, from a remote server to your local machine.

Clone a remote repository to your local machine:

```
git clone <repository-url>  
git clone https://github.com/user/repository.git
```

Git Stage

Git Stage

- **git stage** is a term commonly used to describe the process of adding changes to the staging area in Git, although the actual command to do this is **git add**.
- The staging area (also called the index) is an intermediate space where you can group changes before committing them. This allows you to control exactly which changes are included in a commit.

Git Stage

```
MINGW64:/E/Git/staging-rev

Lenovo@DESKTOP-LU5US00 MINGW64 /E/Git/staging-rev (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 4 commits.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   a.txt
        new file:   b.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../.testing-git.swpx
        ../MyFolder/
        ../commclassroomOP/
        r.txt
        ../staging/
```

Git Log

Git Log

- The Git Log command shows the log the commits for a given repo.

Show Commit History:

```
git log
```

Show a Limited Number of Commits:

```
git log -n 5
```

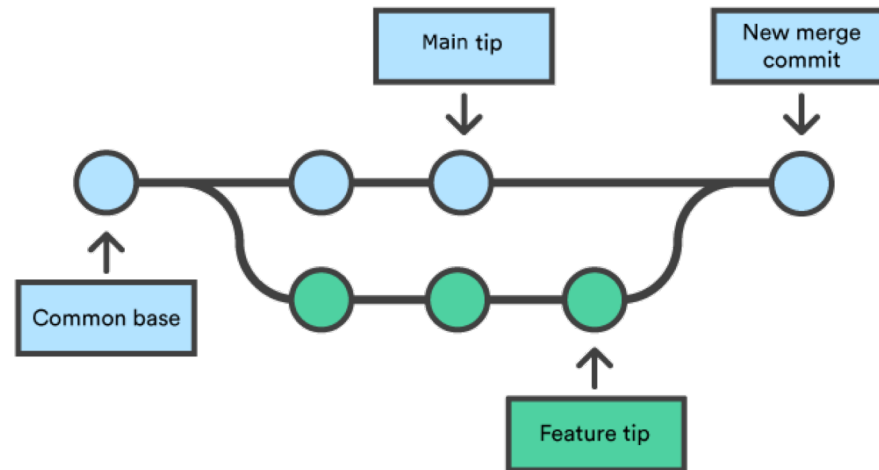
Show Commits by Author:

```
git log --author="Author Name"
```

Branches

What is Branch in Git

- Git branches are effectively a pointer to a snapshot of your changes. When you want to add a new feature or fix a bug—no matter how big or how small—you spawn a new branch to encapsulate your changes.



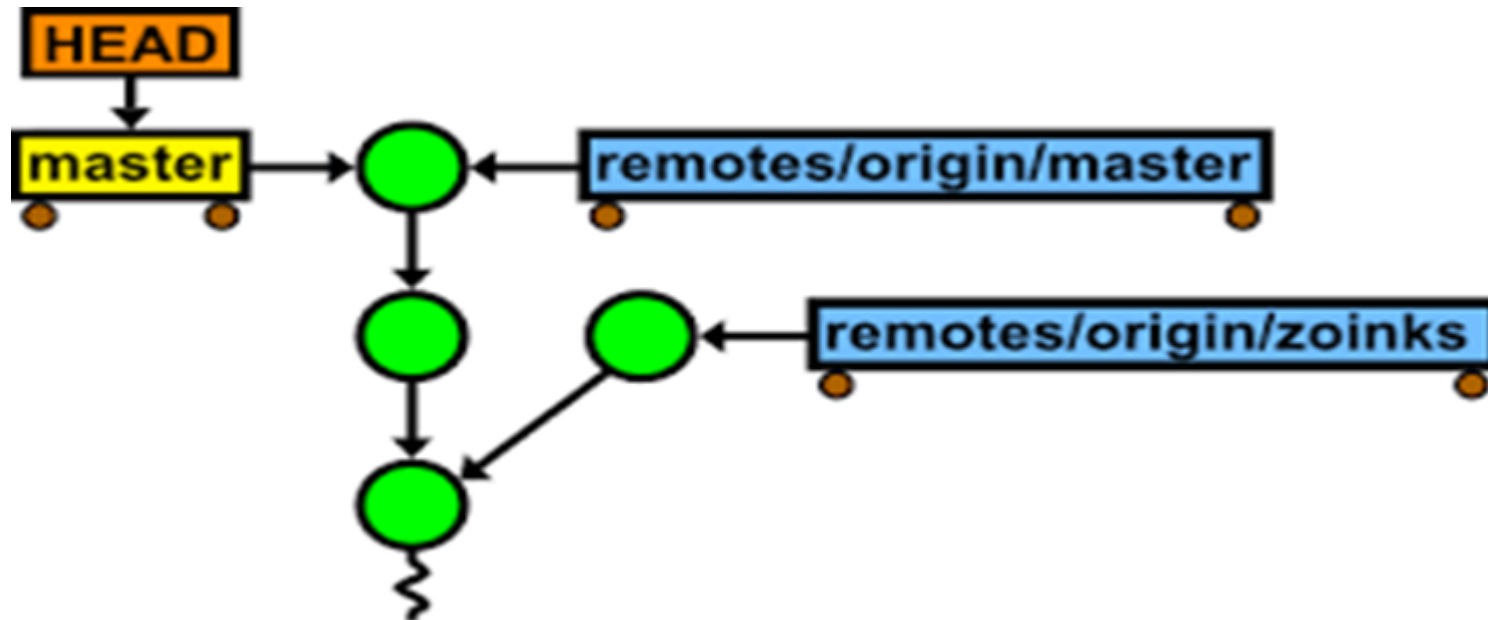
Git Branch

- Git branch command list all the branches we currently have in our project.
- Use “**Git branch < branch name>**” to make a new branch based upon the current HEAD. It does not switch you to that branch.

```
git branch < branch name>
```

What is HEAD -> master

- In stricter terms, HEAD is a dynamic pointer to the commit that reflects the current working directory (tree) state. So, in general, master or main is the first and usually primary branch, while HEAD points to the current commit.



Creating & Switching

- Use git switch with the **-c** flag to create a new branch & switch to it all in one go. -c short of "create"

```
git switch -c <branch name>
```

Deleting & Renaming Branches

Git Branch -m

- **Git branch -m <new-name>** allows us to rename the branch, but in this case we have to be inside the branch.

```
git branch -m <name>
```

Git Branch -d

- **Git branch -d <branch-name>** will delete the Branch, but we cannot delete the branch in which we're currently in.

```
git branch -d <name>
```

Git Checkout

Git Checkout

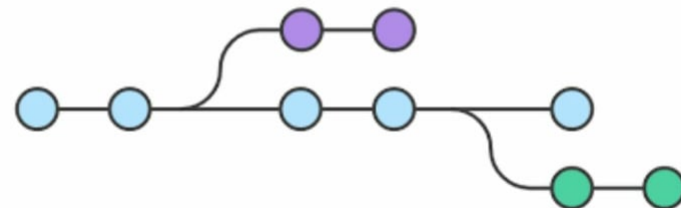
- The **git checkout -b** command is used to create and switch to a new branch in a single step.

```
git checkout -b <new-branch-name>
```

Temp Branches

Temp Branches

- **Purpose:**
 - Used for short-term tasks like experimenting with new features or bug fixes without affecting the main codebase.
- **Creation and Usage:**
 - Easily create with ***git checkout -b temp-branch***, work on it, and delete when done using ***git branch -d temp-branch***.
- **Merging or Discarding:**
 - If the work is successful, merge the changes back to the main branch; otherwise, delete the branch to discard changes.



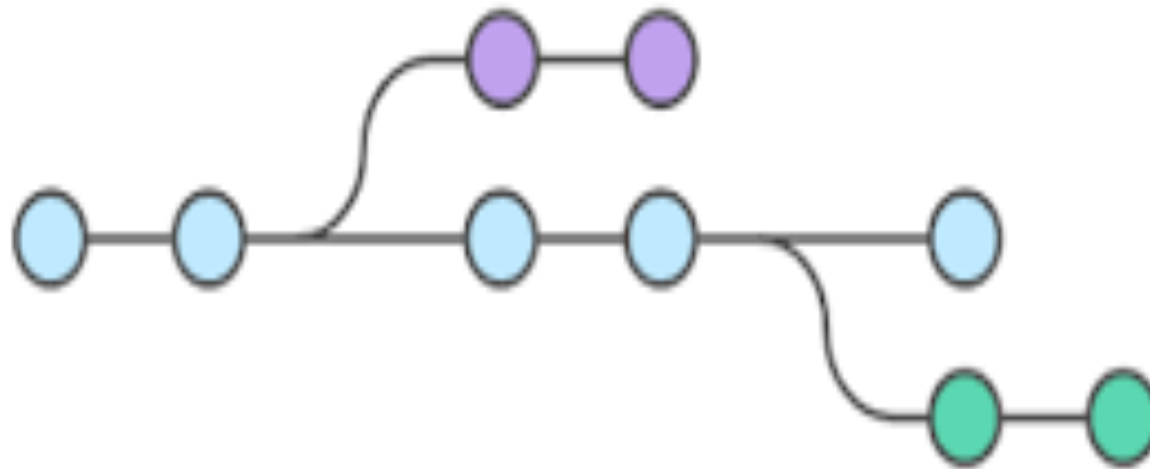
When to Create Branches - Feature Level

- Create a new branch for each feature or enhancement to isolate changes from the main codebase, preventing conflicts and ensuring a clean workflow.
- Use feature branches to allow multiple developers to work on different features simultaneously without interfering with each other's work.
- Creating a branch for each feature enables parallel development, allowing teams to work on multiple features or bug fixes concurrently and independently.

Merging

What Is Merging In Git

- Merging is Git's way of putting a forked history back together again. The `git merge` command lets you take the independent lines of development created by `git branch` and integrate them into a single branch.



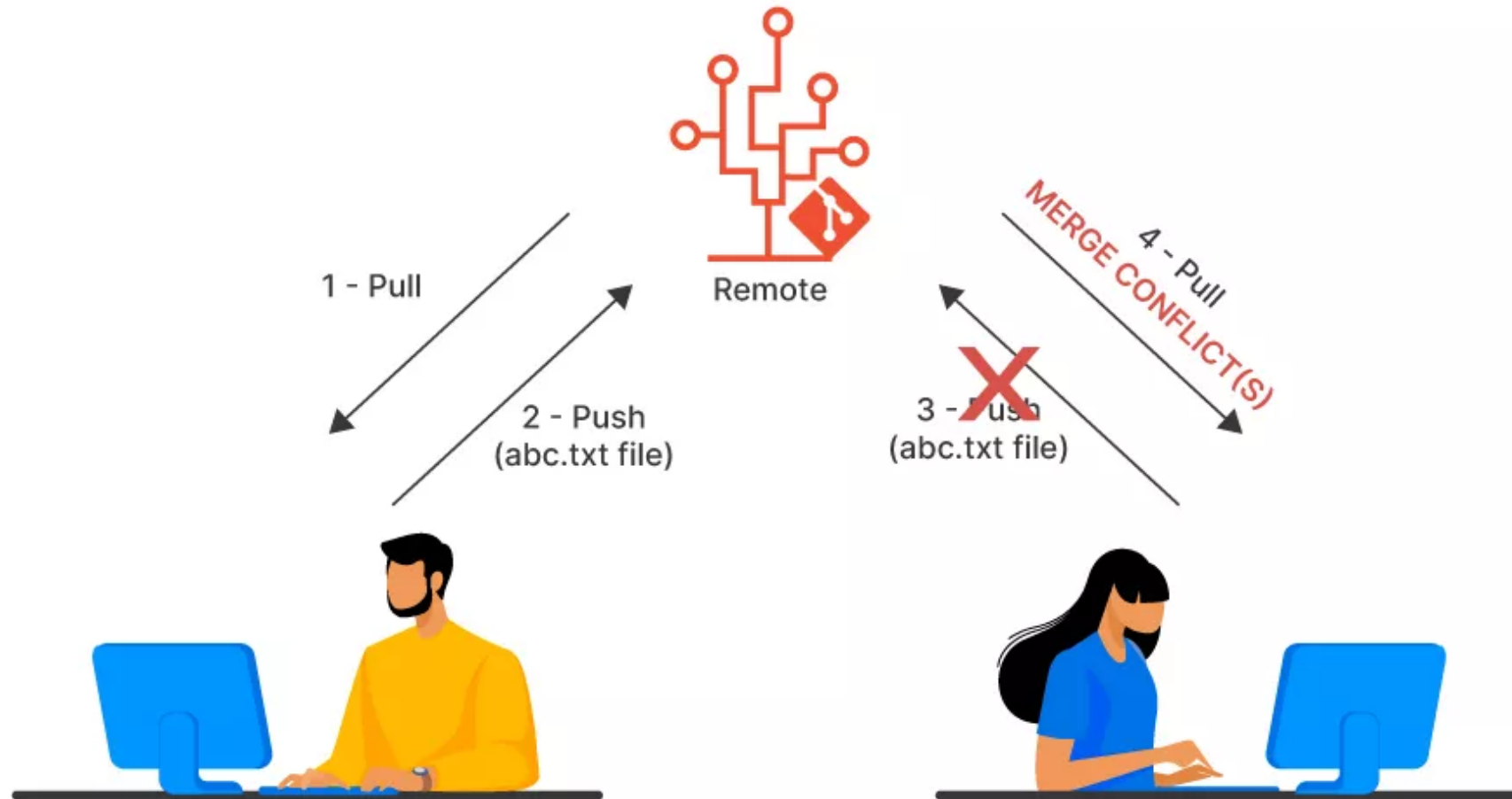
Git Branch -d

- Git **merge** <branch-name> allows us to merge two branches.
- We merge branches, **not** specific commits
- We always merge to the current **HEAD** branch

```
git merge < branch-name>
```

Merge Conflict

Merge Conflict



Resolving Merge Conflicts in Git

1. Create and Modify Branches:

- Create two branches (**`his-branch`** and **`my-branch`**) and make different changes to the same lines in a file (**`abc.txt`**).

Resolving Merge Conflicts in Git

2. Attempt to Merge and Encounter a Conflict:

- Merge ``his-branch`` into ``my-branch``. Since both branches modify the same lines, Git will flag a conflict.
- Git will notify you of the conflict, requiring manual resolution.

Resolving Merge Conflicts in Git

3. Resolve the Conflict

- Open the conflicted file (**abc.txt**) to see the conflicting changes marked by Git.
- Choose how to resolve:
 - Accept Current Change: Keep changes from my-branch.
 - Accept Incoming Change: Keep changes from his-branch.
 - Accept Both Changes: Combine changes from both branches.
- Save the resolved file.

Resolving Merge Conflicts in Git

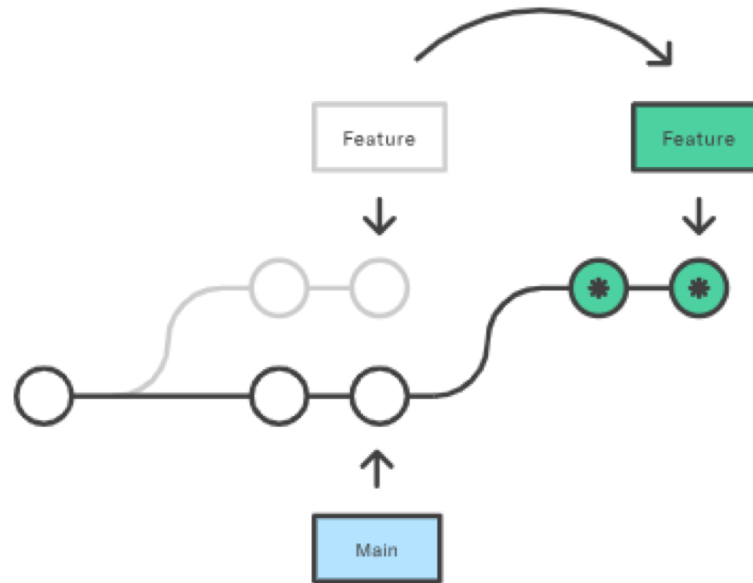
4. Mark the Conflict as Resolved and Commit

- Stage the resolved file using ``git add <file>``.
- Commit the merge with a clear message: ``git commit -m "Merged his-branch into my-branch"``.
- Confirm the successful merge with ``git status`` and ``git log``.

Rebase

What Is Git Rebase?

- Rebasing is the process of moving or combining a sequence of commits to a new base commit. Rebasing is most useful and easily visualized in the context of a feature branching workflow. The general process can be visualized as the following:



Git Rebase

- Rebasing is a common way to integrate upstream changes into your local repository. Pulling in upstream changes with Git merge results in a superfluous merge commit every time you want to see how the project has progressed. On the other hand, rebasing is like saying, “I want to base my changes on what everybody has already done.”

Don't Rebase Public History

- You should never rebase commits once they've been pushed to a public repository. The rebase would replace the old commits with new ones and it would look like that part of your project history abruptly vanished.

git rebase <base>

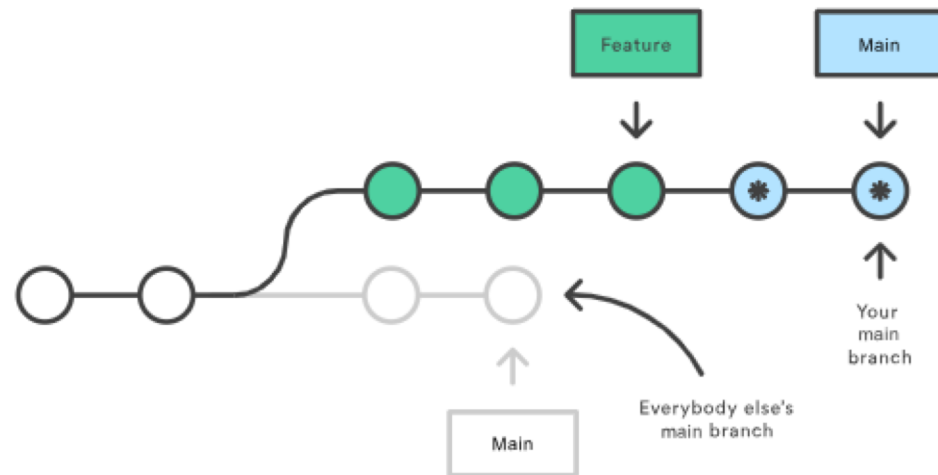
- Git rebase in standard mode will automatically take the commits in your current working branch and apply them to the head of the passed branch

```
git rebase <base>
```

- This automatically rebases the current branch onto <base>, which can be any kind of commit reference (for example an ID, a branch name, a tag, or a relative reference to HEAD).

The Golden Rule Of Rebasing

- Once you understand what rebasing is, the most important thing to learn is when not to do it. The golden rule of git rebase is to never use it on public branches.
- For example, think about what would happen if you rebased main onto your feature branch:



Git Cherry-Picking

Git Cherry-Picking

- Git Cherry-Picking allows you to apply specific commits from one branch to another without merging the entire branch.



Git Cherry-Picking

Select a Commit to Cherry-Pick:

- Identify the commit you want to apply to your current branch using its commit hash.
- Example:

```
git cherry-pick <commit-hash>
```

.gitignore

.gitignore

- The **.gitignore** file in a Git repository specifies files and directories that should be ignored by Git. These files won't be tracked, staged, or committed, which is useful for excluding temporary files, build artifacts, and other files that should not be part of the repository.

SSH Keys

What Is A Git SSH Key?

- An SSH key is an access credential used for the SSH (secure shell) network protocol, which provides authenticated and encrypted communication between machines over unsecured networks.
- It consists of a pair of keys: a public key and a private key. The public key is shared with remote parties to encrypt data, acting like a lock, while the private key is kept secure and used to decrypt the data, functioning as the key.
- SSH keys are commonly used for remote file transfer, network management, and remote operating system access

SSH Keys

- Using the SSH protocol, you can connect and authenticate to remote servers and services. With SSH keys, you can connect to GitHub without supplying your username and personal access token at each visit. You can also use an SSH key to sign commits.

Check For Existing SSH Key

- Open Git Bash.
- Enter **ls -al ~/.ssh** to see if existing SSH keys are present.

\$ ls -al ~/.ssh

Lists the files in your .ssh directory, if they exist

Generating A New SSH Key

- Open Git Bash.
- Paste the text below, replacing the email used in the example with your GitHub email address.

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Adding Your SSH Key To The SSH-agent

- The ssh-agent is another program that is part of the SSH tool suite. The ssh-agent is responsible for holding private keys. Think of it like a keychain. In addition to holding private keys it also brokers requests to sign SSH requests with the private keys so that private keys are never passed around unsecured.
- Before adding the new SSH key to the ssh-agent first ensure the ssh-agent is running by executing:

```
eval "$(ssh-agent -s)"
```

- Once the ssh-agent is running the following command will add the new SSH key to the local SSH agent.

```
ssh-add -K /Users/you/.ssh/id_rsa
```

Thank You